

Evidencias Semana 8

Pixel Store

Actividad: Mejorando funcionalidades clave en el eCommerce con React.

Asignatura: Desarrollo Frontend I · PFY2201.

Estudiante: Juan Carlos Osega.

Fecha de revisión: 5 de octubre de 2026.

Estado: implementación y evidencias locales verificadas; publicación de la versión React pendiente.

Actualización del entregable anterior

El informe anterior, ubicado en [Frontend/Actividad/Juan_Osega_PFY2201_Evidencias_Semana6.pdf](#), documentaba una aplicación con JavaScript y nueve capturas: inicio, catálogo, filtros, búsqueda, carrito, vista móvil y publicación. Esta versión acredita la gestión de estados, efectos y renderizado condicional de Semana 8 y conserva la identidad visual de la tienda.

Se mantiene el informe de Semana 6 como antecedente. Sus capturas de publicación no acreditan el despliegue de la versión React.

Resultado de la verificación

- Compilación de producción completada mediante [npm run build](#).
- Tres pruebas funcionales aprobadas mediante [npm test](#).
- Auditoría de dependencias sin vulnerabilidades reportadas mediante [npm audit](#).
- Dos capturas funcionales actuales incluidas en este informe.

Las capturas se obtuvieron del sitio compilado servido localmente. La evaluación CL también requiere revisar el código y comprobar que la versión actual esté publicada y accesible.

Correspondencia con la pauta

1. Gestión de estados · 25 puntos

`src/App.jsx` administra con `useState` el catálogo (`productos`), el carrito (`carrito`), la búsqueda, la categoría seleccionada, la carga, los errores y el reintento. Las actualizaciones del carrito crean nuevos objetos y arreglos, sin modificar el estado anterior. El contador suma las unidades y el total considera precio por cantidad.

2. Manejo de efectos · 20 puntos

`useEffect` carga `assets/data/productos.json` mediante `fetch`, valida la respuesta y actualiza el catálogo. El efecto depende del estado de reintento. La función de limpieza cancela la solicitud pendiente con `AbortController`. La carga y los errores tienen mensajes visibles.

3. Renderizado condicional · 20 puntos

`src/components/Producto.jsx` cambia el botón de «Agregar» a «En el carrito · +1». `src/components/Carrito.jsx` muestra el mensaje de carrito vacío o los productos seleccionados y deshabilita el vaciado cuando no hay productos. El catálogo muestra mensajes de carga, error y búsqueda sin resultados según el estado.

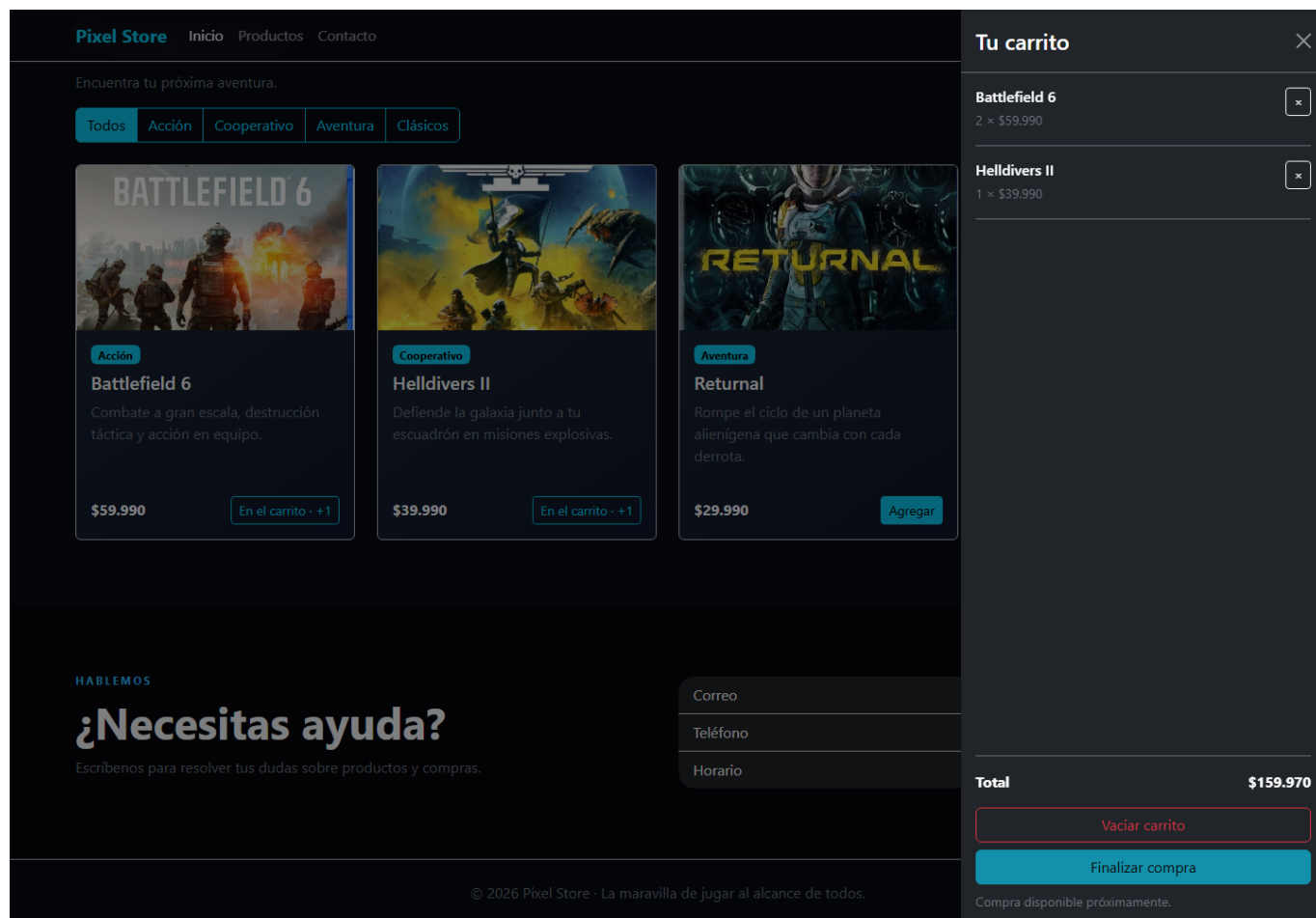
4. Organización y claridad · 15 puntos

`src/main.jsx` inicia la aplicación. `App.jsx` concentra los estados compartidos y comunica datos y acciones mediante props. Los componentes reutilizables de producto y carrito separan la presentación. Los estilos, imágenes y datos mantienen carpetas propias. Los comentarios son breves, precisos y explican la lógica sin referencias a entidades. La navegación, el carrusel, los colores y las imágenes conservan el estilo anterior.

5. Publicación · 20 puntos

La compilación genera `dist` con rutas relativas. `npm run deploy` está configurado para publicar ese contenido en `gh-pages`. El código está en la rama local `exp3-s8`. La publicación del código actualizado y del sitio sigue pendiente; este criterio no se acredita todavía como CL.

Evidencia 1 · Catálogo y carrito



Catálogo y carrito con productos seleccionados

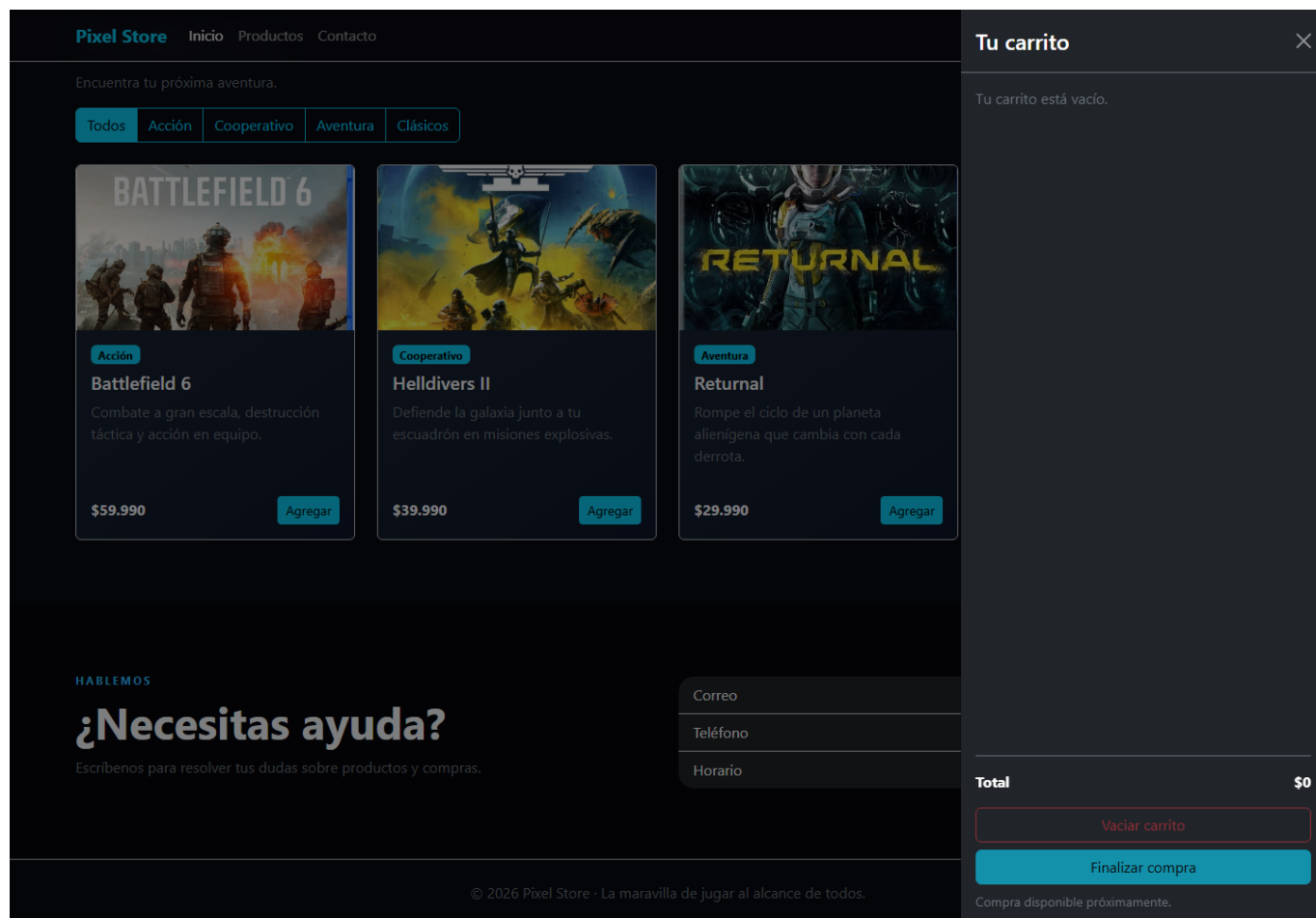
El catálogo muestra productos cargados desde JSON. Se agregaron dos unidades de Battlefield 6 y una de Helldivers II. El carrito presenta cantidades, eliminación y un total de \$159.970. Los botones de los productos seleccionados cambian a «En el carrito · +1».

La prueba comprueba además el contador de tres unidades. Después de eliminar Battlefield 6, comprueba una unidad y un total de \$39.990.

Criterios relacionados: gestión de estados, carga dinámica y renderizado condicional. La utilización de los hooks se acredita mediante el código señalado en la página anterior.

Origen: versión de producción ejecutada localmente. Esta captura no constituye una evidencia de publicación.

Evidencia 2 · Carrito vacío



Carrito vacío después de eliminar los productos

Después de eliminar un producto y vaciar el carrito, aparece «Tu carrito está vacío», el total vuelve a \$0 y «Vaciar carrito» queda deshabilitado. Los botones del catálogo vuelven a «Agregar».

La prueba comprueba además que el contador vuelve a cero. La comparación con la evidencia anterior muestra la actualización de la interfaz según el estado.

Criterios relacionados: gestión del carrito y renderizado condicional de mensajes y botones.

Origen: versión de producción ejecutada localmente.

Publicación y entrega

Enlaces

Repositorio identificado: <https://github.com/JotaXIII/Frontend-1>

Rama de código actual: [exp3-s8](#), disponible localmente; pendiente de publicación.

Rama de despliegue: [gh-pages](#).

URL esperada del sitio: <https://jotaxiii.github.io/Frontend-1/>

La URL del sitio se deduce del repositorio. Confirma el enlace definitivo en la configuración de Pages después del despliegue. No se verificó en esta revisión la publicación de la versión React.

Mínimo de capturas propuesto: tres

1. Catálogo con el carrito abierto y productos agregados: cantidades, total y al menos un botón «En el carrito +1». Incluida como evidencia 1.
2. Carrito después de eliminar los productos: mensaje de vacío, total \$0 y botones «Agregar». Incluida como evidencia 2.
3. Configuración de Pages después de publicar: mensaje de sitio disponible, URL, rama [gh-pages](#) y carpeta raíz. Pendiente.

No se necesitan capturas adicionales de inicio, filtros, búsqueda, diseño móvil o errores para cubrir los entregables explícitos. Conserva las dos primeras o reemplázalas por las mismas vistas del sitio publicado. Si las tomas nuevamente, incluye la barra de direcciones para identificar la versión desplegada.

Las tres capturas complementan el código y los enlaces; no sustituyen una publicación funcional ni garantizan por sí solas una calificación CL.

Pendientes antes de entregar

- Publicar la rama de código para que el proyecto React pueda revisarse en el repositorio público.
- Publicar [dist](#) en [gh-pages](#) y comprobar el catálogo, las imágenes y el carrito en el sitio remoto.
- Agregar la tercera captura al informe y actualizar el estado de publicación una vez verificado.
- Entregar en AVA el PDF actualizado, el enlace del repositorio y el enlace confirmado del despliegue.

Archivos de apoyo

[README.md](#) contiene la estructura, los requisitos y los comandos de ejecución, compilación, pruebas y publicación. [tests/tienda.spec.js](#) verifica el carrito, los filtros, la vista móvil y el reintento después de un error.

El informe puede regenerarse con [python scripts/generar_evidencias.py](#). Requiere el paquete [reportlab](#).